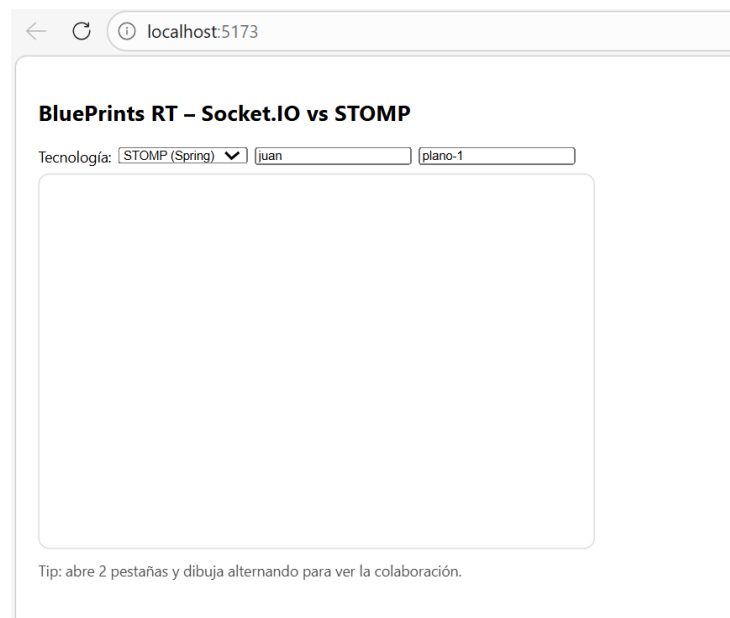


1. Puesta en Marcha

Se optó por la opción A: Socket.IO con Node.js como backend de tiempo real, ya que permite gestionar salas de forma nativa y tiene integración directa con el frontend React a través de la librería socket.io-client.

1a. Backend RT Socket.IO (Node.js) en :3001

Se clonó el repositorio guía y se instalaron las dependencias. El proyecto frontend ya contaba con su propio package.json, por lo que se mantuvo como repositorio independiente bajo la organización de GitHub del equipo:



En el repositorio del backend se ejecutaron los comandos de instalación y arranque:

```
PS C:\Users\robin\downloads\LAB07_ARSW_BACKEND> npm i
To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\robin\downloads\LAB07_ARSW_BACKEND> npm run dev

> backend-socketio-node@0.1.0 dev
> node server.js

Socket.IO up on :3001
█
```

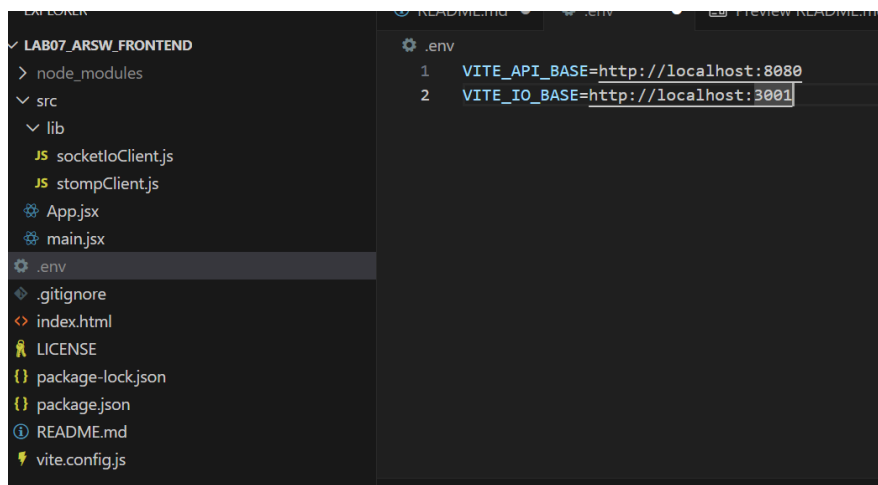
Se verificó el endpoint REST con curl para confirmar que el servidor responde correctamente:

```
PS C:\Users\robin\downloads\LAB07_ARSW_BACKEND> curl http://localhost:3001/api/blueprints/juan/plano-1
[?] Ayuda (el valor predeterminado es "N"): S

StatusCode      : 200
StatusDescription : OK
Content         : {"author":"juan","name":"plano-1","points":[{"x":10,"y":10},{"x":40,"y":50}]}
RawContent      : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 77
                  Content-Type: application/json; charset=utf-8
                  Date: Sat, 14 Mar 2026 22:37:35 GMT
                  ...
Forms           : {}
Headers         : {[Access-Control-Allow-Origin, *], [Connection, keep-alive], [Keep-Alive, timeout=5],
                  [Content-Length, 77]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 77
```

1b. Frontend corriendo en :5173

Se creó el archivo .env.local en la raíz del proyecto frontend apuntando al backend CRUD en :8080 y al backend Socket.IO en :3001:



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure for LAB07_ARSW_FRONTEND, including node_modules, src, lib, App.jsx, main.jsx, .env, .gitignore, index.html, LICENSE, package-lock.json, package.json, README.md, and vite.config.js. The .env file is selected and its contents are displayed in the code editor. The .env file contains two lines: VITE_API_BASE=http://localhost:8080 and VITE_IO_BASE=http://localhost:3001.

```
.env
1 VITE_API_BASE=http://localhost:8080
2 VITE_IO_BASE=http://localhost:3001
```

Se cambió el estado inicial del selector de tecnología de STOMP a Socket.IO para que la aplicación arranque directamente en el modo correcto:

```
App.jsx > App
import { useEffect, useRef, useState } from 'react'
import { createStompClient, subscribeBlueprint } from './lib/stompClient.js'
import { createSocket } from './lib/socketIoClient.js'

const API_BASE = import.meta.env.VITE_API_BASE ?? 'http://localhost:8080' // Spring
const IO_BASE = import.meta.env.VITE_IO_BASE ?? 'http://localhost:3001' // Node/Socket.IO

export default function App() {
  const [tech, setTech] = useState('socketio')
  const [author, setAuthor] = useState('juan')
  const [name, setName] = useState('plano-1')
  const canvasRef = useRef(null)
```

2. Protocolo Socket.IO Implementado

Se implementaron los tres eventos del protocolo Socket.IO: join-room al abrir un plano, draw-event al hacer clic en el canvas, y blueprint-update para recibir y dibujar los puntos de otras pestañas.

Durante la implementación se identificaron y corrigieron dos bugs críticos:

- Bug 1: El clic enviaba el punto al servidor, pero nunca dibujaba localmente. El componente solo pintaba al recibir blueprint-update, pero el servidor hace broadcast únicamente a los demás clientes, excluyendo al emisor. La solución fue dibujar el punto localmente de inmediato antes de emitir el evento.
- Bug 2: El evento join-room se emitía antes de que el socket estuviera conectado, por lo que la sala nunca se unía correctamente. La solución fue mover el emit dentro del callback del evento connect.

Los cambios se aplicaron en App.jsx:

```
// Dibuja todos los puntos acumulados
function redraw() {
  const ctx = canvasRef.current?.getContext('2d')
  if (!ctx) return
  ctx.clearRect(0, 0, 600, 400)
  ctx.strokeStyle = '#2563eb'
  ctx.lineWidth = 2
  const pts = pointsRef.current
  if (pts.length === 0) return
  ctx.beginPath()
  ctx.moveTo(pts[0].x, pts[0].y)
  pts.forEach(p => ctx.lineTo(p.x, p.y))
  ctx.stroke()
  pts.forEach(p => {
    ctx.beginPath()
    ctx.arc(p.x, p.y, 4, 0, Math.PI * 2)
    ctx.fillStyle = '#2563eb'
    ctx.fill()
  })
}
```

```
// Carga el estado inicial
useEffect(() => {
  const base = tech === 'stomp' ? API_BASE : IO_BASE
  pointsRef.current = []
  fetch(`${base}/api/blueprints/${author}/${name}`)
    .then(r => r.json())
    .then(bp => appendPoints(bp.points ?? []))
    .catch(err => console.error('GET inicial falló:', err))
}, [tech, author, name])

// Configura la conexión WebSocket
useEffect(() => {
  unsubRef.current?.()
  unsubRef.current = null
  stompRef.current?.deactivate?.()
  stompRef.current = null
  socketRef.current?.disconnect?.()
  socketRef.current = null

  if (tech === 'stomp') {
    const client = createStompClient(API_BASE)
    stompRef.current = client
    client.onConnect = () => {
      unsubRef.current = subscribeBlueprint(client, author, name, upd => {
        appendPoints(upd.points ?? [])
      })
    }
    client.activate()
  } else {
    const s = createSocket(IO_BASE)
    socketRef.current = s
    const room = `blueprints.${author}.${name}`
    s.on('connect', () => {
      console.log('Socket.IO conectado, joining room:', room)
      s.emit('join-room', room)
    })
    s.on('blueprint-update', upd => {
      console.log('blueprint-update recibido:', upd)
      appendPoints(upd.points ?? [])
    })
  }

  return () => {
    unsubRef.current?.()
    unsubRef.current = null
    stompRef.current?.deactivate?.()
    socketRef.current?.disconnect?.()
  }
}, [tech, author, name])
```

```
// Maneja clicks en el canvas para dibujar y enviar eventos
function onClick(e) {
  const rect = e.target.getBoundingClientRect()
  const point = {
    x: Math.round(e.clientX - rect.left),
    y: Math.round(e.clientY - rect.top),
  }

  appendPoints([point])

  if (tech === 'stomp' && stompRef.current?.connected) {
    stompRef.current.publish({
      destination: '/app/draw',
      body: JSON.stringify({ author, name, point }),
    })
  } else if (tech === 'socketio' && socketRef.current?.connected) {
    const room = `blueprints.${author}.${name}`
    socketRef.current.emit('draw-event', { room, author, name, point })
  }
}

return (
  <div style={{ fontFamily: 'Inter, system-ui', padding: 16, maxWidth: 900 }}>
    <h2>BluePrints RT <span>Socket.IO vs STOMP</span></h2>
    <div style={{ display: 'flex', gap: 8, alignItems: 'center', marginBottom: 8 }}>
      <label>Tecnología:</label>
      <select value={tech} onChange={e => setTech(e.target.value)}>
        <option value="stomp">STOMP (Spring)</option>
        <option value="socketio">Socket.IO (Node)</option>
      </select>
      <input value={author} onChange={e => setAuthor(e.target.value)} placeholder="autor" />
      <input value={name} onChange={e => setName(e.target.value)} placeholder="plano" />
    </div>
    <canvas
      ref={canvasRef}
      width={600}
      height={400}
      style={{ border: '1px solid #ddd', borderRadius: 12, cursor: 'crosshair' }}
      onClick={onClick}
    />
    <p style={{ opacity: .7, marginTop: 8 }}>
      Tip: abre 2 pestañas y dibuja alternando para ver la colaboración.
    </p>
  </div>
)
```

- Backend: server.js

Gracias a la validación de payloads agregada, el servidor registra en consola cada conexión y evento recibido. Los logs confirman que las salas se crean correctamente y que ningún payload inválido llega al broadcast:

```
// Socket.IO
const server = http.createServer(app)
const io = new Server(server, { cors: { origin: '*' } })

io.on('connection', (socket) => {
  socket.on('join-room', (room) => {
    if (!room || typeof room !== 'string') {
      console.warn('join-room ignorado: room inválido', room)
      return
    }
    socket.join(room)
    console.log(`join-room: ${room}`)
  })
  socket.on('draw-event', ({ room, point, author, name }) => {
    if (!room || !author || !name) {
      console.warn('draw-event ignorado: faltan campos obligatorios')
      return
    }
    if (typeof point?.x !== 'number' || typeof point?.y !== 'number') {
      console.warn('draw-event ignorado: punto inválido', point)
      return
    }
    const list = getAuthorBlueprints(author)
    const bp = list.find(b => b.name === name)
    if (bp) bp.points.push(point)
    socket.to(room).emit('blueprint-update', { author, name, points: [point] })
  })
})

const PORT = process.env.PORT || 3001
server.listen(PORT, () => console.log(`Socket.IO up on :${PORT}`))
```

- Comprobación:

```
> node server.js

Socket.IO up on :3001
join-room: blueprints.juan.plano-1
join-room: blueprints.juan.plano-1
join-room: blueprints.juan.Robin
join-room: blueprints.juan.plano-1
join-room: blueprints.juan.Robin
join-room: blueprints.juan.plano-1
join-room: blueprints.juan.Robin
PS C:\Users\robin\downloads\LAB07_ARSW_BACKEND> npm run dev

> backend-socketio-node@0.1.0 dev
> node server.js

Socket.IO up on :3001
join-room: blueprints.juan.Robin
join-room: blueprints.juan.plano-1
join-room: blueprints.juan.Robin
PS C:\Users\robin\downloads\LAB07_ARSW_BACKEND> npm run dev

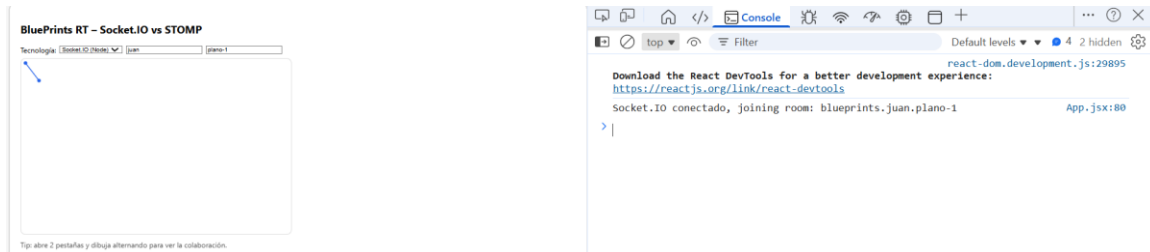
> backend-socketio-node@0.1.0 dev
> node server.js

Socket.IO up on :3001
join-room: blueprints.juan.Robin
join-room: blueprints.juan.plano-1
Socket.IO up on :3001
join-room: blueprints.juan.Robin
join-room: blueprints.juan.plano-1
```

3. Casos de Prueba Mínimos

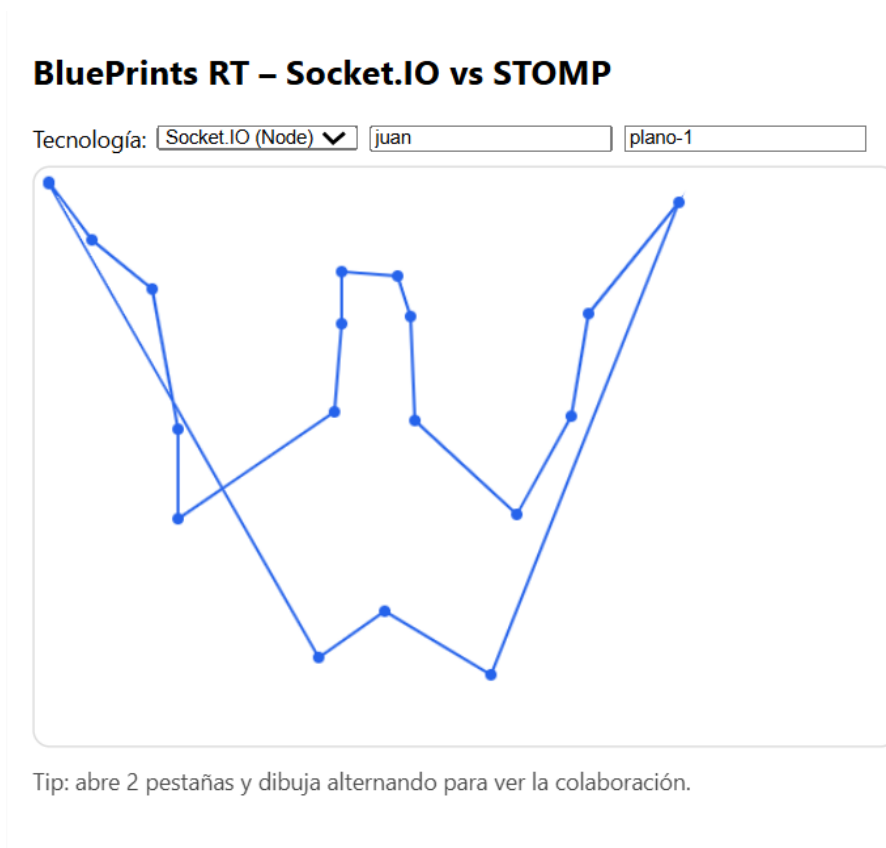
3a. Estado inicial: Canvas carga puntos del GET

Al seleccionar un plano, el frontend realiza un GET al endpoint `/api/blueprints/:author/:name` y dibuja los puntos existentes en el canvas antes de habilitar la edición:



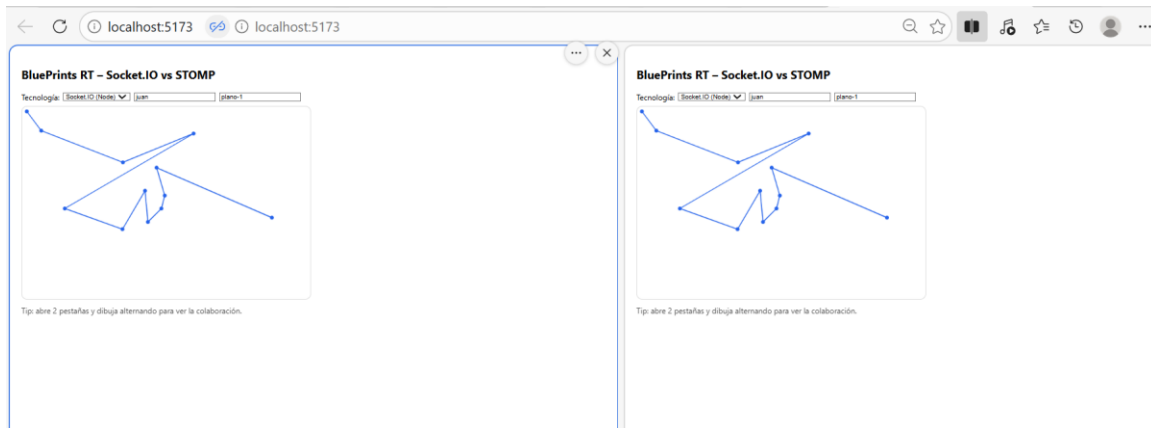
3b. Dibujo local: Clic agrega punto y redibuja

Cada clic en el canvas registra las coordenadas, dibuja el punto localmente de inmediato y envía el evento `draw-event` al servidor para que los demás clientes lo reciban vía broadcast:



3c. RT multi-pestaña: Trazo se replica en tiempo real

Con dos pestañas abiertas en el mismo plano con el mismo autor y nombre, los puntos dibujados en una pestaña aparecen en la otra en menos de 1 segundo, demostrando la colaboración en tiempo real a través del canal Socket.IO:



3d. CRUD: Create / Save / Delete

Para habilitar el CRUD completo se extendió tanto el backend como el frontend:

- Backend: server.js

Se agregaron los endpoints POST, PUT y DELETE al servidor Express, junto con un GET por autor que incluye el total de puntos calculado con reduce:

```
// REST CRUD
app.get('/api/blueprints', (req, res) => {
  const { author } = req.query
  if (!author) return res.status(400).json({ error: 'author requerido' })
  const list = getAuthorBlueprints(author)
  const total = list.reduce((sum, bp) => sum + bp.points.length, 0)
  res.json({ author, blueprints: list, totalPoints: total })
})

// GET /api/blueprints/:author/:name, para los puntos del plano
app.get('/api/blueprints/:author/:name', (req, res) => {
  const { author, name } = req.params
  const bp = getAuthorBlueprints(author).find(b => b.name === name)
  if (!bp) return res.status(404).json({ error: 'No encontrado' })
  res.json(bp)
})

// POST /api/blueprints, crea un nuevo plano
app.post('/api/blueprints', (req, res) => {
  const { author, name, points = [] } = req.body
  if (!author || !name) return res.status(400).json({ error: 'author y name requeridos' })
  if (!db[author]) db[author] = []
  if (db[author].find(b => b.name === name)) {
    return res.status(409).json({ error: 'Ya existe' })
  }
  const bp = { author, name, points }
  db[author].push(bp)
  res.status(201).json(bp)
})

// PUT /api/blueprints/:author/:name, actualiza los puntos
app.put('/api/blueprints/:author/:name', (req, res) => {
  const { author, name } = req.params
  const list = getAuthorBlueprints(author)
  const idx = list.findIndex(b => b.name === name)
  if (idx === -1) return res.status(404).json({ error: 'No encontrado' })
  list[idx].points = req.body.points ?? list[idx].points
  res.json(list[idx])
})

// DELETE /api/blueprints/:author/:name, elimina los puntos del plano
app.delete('/api/blueprints/:author/:name', (req, res) => {
  const { author, name } = req.params
  if (!db[author]) return res.status(404).json({ error: 'No encontrado' })
  const before = db[author].length
  db[author] = db[author].filter(b => b.name !== name)
  if (db[author].length === before) {
    return res.status(404).json({ error: 'No encontrado' })
  }
  res.status(204).end()
})
```

Frontend: blueprintsApi.js

Se creó el módulo blueprintsApi.js para encapsular todas las llamadas REST, manteniendo el App.jsx limpio y separando responsabilidades:

```
export function createApi(baseUrl) {
  const h = { 'Content-Type': 'application/json' }

  return {
    async list(author) {
      const r = await fetch(`${baseUrl}/api/blueprints?author=${author}`)
      return r.json()
    },
    async get(author, name) {
      const r = await fetch(`${baseUrl}/api/blueprints/${author}/${name}`)
      return r.json()
    },
    async create(author, name) {
      const r = await fetch(`${baseUrl}/api/blueprints`, {
        method: 'POST', headers: h,
        body: JSON.stringify({ author, name, points: [] })
      })
      if (!r.ok) throw await r.json()
      return r.json()
    },
    async save(author, name, points) {
      const r = await fetch(`${baseUrl}/api/blueprints/${author}/${name}`, {
        method: 'PUT', headers: h,
        body: JSON.stringify({ points })
      })
      if (!r.ok) throw new Error('Error al guardar')
      return r.json()
    },
    async remove(author, name) {
      const r = await fetch(`${baseUrl}/api/blueprints/${author}/${name}`, {
        method: 'DELETE'
      })
      if (!r.ok) throw new Error('Error al eliminar')
    },
  }
}
```

Frontend App.jsx, handlers CRUD

Se integraron los handlers handleCreate, handleSave y handleDelete en el componente, cada uno consume el módulo blueprintsApi y refresca la lista del autor al completarse:

```
// Maneja la creacion de un nuevo plano
async function handleCreate() {
  if (!newName.trim()) return
  try {
    await api.create(author, newName.trim())
    setStatus('✅ Creado: ${newName}')
    setNewName('')
    loadList()
  } catch (e) { setStatus('❌ ${e.error ?? 'Error al crear'}') }
}

// Maneja el guardado del plano actual
async function handleSave() {
  try {
    await api.save(author, name, pointsRef.current)
    setStatus('✅ Guardado')
    loadList()
  } catch { setStatus('❌ Error al guardar') }
}

// Maneja la eliminacion del plano actual
async function handleDelete(bpName) {
  try {
    await api.remove(author, bpName)
    setStatus('❌ Eliminado: ${bpName}')
    if (bpName === name) { pointsRef.current = []; redraw() }
    loadList()
  } catch { setStatus('❌ Error al eliminar') }
}
```


Frontend con Interfaz actualizada

Se rediseñó la interfaz en dos paneles: el panel izquierdo con la tabla de planos, total de puntos, campo de creación y botón de guardar y el panel derecho con el canvas interactivo:

```
return (
  <div style={{ fontFamily: 'Inter, system-ui', padding: 16, display: 'flex', gap: 24, maxWidth: 1100 }}>
    /* Panel izquierdo - CRUD */
    <div style={{ width: 300, flexShrink: 0 }}>
      <h2 style={{ margin: 0 }}>Blueprints RT</h2>
      <div style={{ display: 'flex', gap: 6, margin: 10, alignItems: 'center' }}>
        <label>Tecnología:</label>
        <select value={tech} onChange={e => setTech(e.target.value)}>
          <option value="socketio">Socket.IO (Node)</option>
          <option value="stomp">STOMP (Spring)</option>
        </select>
      </div>
      <div style={{ display: 'flex', gap: 6, margin: 12, alignItems: 'center' }}>
        <label>Autor:</label>
        <input value={author} onChange={e => setAuthor(e.target.value)} placeholder="autor" style={{ width: 120 }} />
      </div>
      <h3 style={{ margin: 4 }}>Planos de {author}</h3>
      <p style={{ margin: '0 0 8px', opacity: .7, fontSize: 13 }}>Total puntos: <strong>{total}</strong></p>
      <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 13 }}>
        <thead>
          <tr style={{ background: '#f1f5f9' }}>
            <th style={{ textAlign: 'left', padding: '4px 8px' }}>Nombre</th>
            <th style={{ textAlign: 'right', padding: '4px 8px' }}>Pts</th>
          </tr>
        </thead>
        <tbody>
          {list.map(bp => (
            <tr key={bp.name}>
              <td style={{ background: bp.name === name ? '#e0e0e0' : 'transparent', cursor: 'pointer' }}>
                <div style={{ padding: '4px 8px' }}>{bp.name}</div>
              <td style={{ padding: '4px 8px', textAlign: 'right' }}>{bp.points.length}</td>
              <td style={{ padding: '4px 8px', textAlign: 'center' }}>
                <button onClick={e => { e.stopPropagation(); handleDelete(bp.name) }}>
                  ✖
                </button>
              </td>
            </tr>
          ))}
          {list.length === 0 && (
            <tr><td colspan={3} style={{ padding: 8, opacity: .5 }}>Sin planos</td></tr>
          )}
        </tbody>
      </table>
      /* Crear nuevo plano */
      <div style={{ display: 'flex', gap: 6, margin: 12 }}>
        <input value={newName} onChange={e => setNewName(e.target.value)}
          placeholder="nuevo nombre" style={{ flex: 1 }} />
        <button onClick={handleCreate}>➕</button>
      </div>
    </div>

```

```
    /* Guardar plano activo */
    <button onClick={handleSave}>
      <div style={{ margin: 8, width: '100%', background: '#25c328', color: 'white',
        border: 'none', padding: '7px 0', borderRadius: 6, cursor: 'pointer', fontSize: 14 }}>
        Guardar plano
      </div>
    </button>
    {status && <p style={{ margin: 8, fontSize: 13 }}>{status}</p>}
  </div>
  /* Panel derecho - Canvas */
  <div>
    <p style={{ margin: '0 0 6px', opacity: .7, fontSize: 13 }}>Plano activo: <strong>{author}/{name}</strong></p>
    <canvas ref={canvasRef} width={600} height={400}>
      <div style={{ border: '1px solid #ccc', borderRadius: 12, cursor: 'crosshair', display: 'block' }}
        onClick={onClick} />
    <p style={{ opacity: .7, margin: 8, fontSize: 13 }}>Tip: abre 2 postas y dibuja alternando para ver la colaboración.</p>
  </div>
)
}
```

Evidencia del CRUD

- Creat: Se crea el plano correctamente y aparece en la tabla:

BluePrints RT

Tecnología:

Autor:

Planos de juan

Total puntos: 32

Nombre	Pts	
plano-1	31	
plano-2	1	
Robin	0	



 Guardar plano

✓ Creado: Robin

Plano activo: juan/Robin



Tip: abre 2 pestañas y dibuja alternando para ver la colaboración.

- Save: Al guardar, el contador de puntos del plano se incrementa en la tabla:

← ↻ ⓘ localhost:5173 🔍 ☆

BluePrints RT

Tecnología:

Autor:

Planos de juan

Total puntos: 49

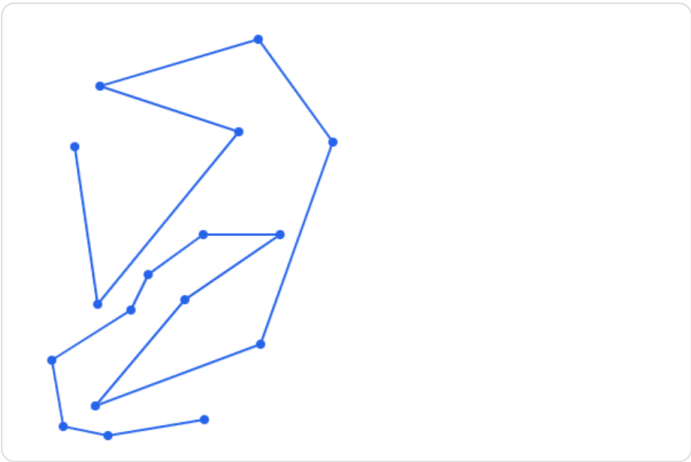
Nombre	Pts	
plano-1	31	
plano-2	1	
Robin	17	



 Guardar plano

✓ Guardado

Plano activo: juan/Robin



Tip: abre 2 pestañas y dibuja alternando para ver la colaboración.

- Delete: Al presionar el ícono de papelera el plano desaparece de la lista:

BluePrints RT

Tecnología: Socket.IO (Node) ▼

Autor:

Planos de juan

Total puntos: 49

Nombre	Pts	
plano-1	31	
plano-2	1	
Robin	17	

+

Guardar plano

☒ Guardado

Plano activo: **juan/Robin**

Tip: abre 2 pestañas y dibuja alternando para ver la colaboración.

- Confirmación de eliminación con el mensaje de estado confirma la operación:

BluePrints RT

Tecnología: Socket.IO (Node) ▼

Autor:

Planos de juan

Total puntos: 32

Nombre	Pts	
plano-1	31	
plano-2	1	

+

Guardar plano

Eliminado: Robin

Plano activo: **juan/Robin**

Tip: abre 2 pestañas y dibuja alternando para ver la colaboración.